

One-Page Explanation — Resume to Job-Description Fit Scorer

1. Design parameter — calibration strength (15%)

I chose to compress the LLM's 0–100 criterion scores by 15% toward the midpoint of 50: $50 + 0.85 \times (\text{raw} - 50)$. I chose this because a small wording difference should not create a very large final-score gap. The scoring prompt also uses temperature 0 and explicit 0/25/50/75/100 anchors. The calibration sample contains two intentionally similar resumes plus one stronger resume. The target is that the two similar resumes remain within a 15-point overall gap. The 15-point limit is a project guardrail, not a universal hiring standard.

2. Failure actually observed

During testing, an unreadable/non-text PDF caused the parser to return no usable text. Scoring an empty string would have produced a misleading result. I changed the parser/API flow so an empty extraction becomes a controlled validation error instead of being sent to the LLM. A production follow-up would add OCR for scanned PDFs.

3. Metric tracked

I tracked end-to-end API latency in milliseconds and expose it in every successful response. This shows the cost of the full pipeline, which includes criterion extraction plus separate LLM scoring calls. I also track the A/B calibration gap in the sample script to monitor score stability.

4. Not finished / next step

I did not build OCR, a labeled evaluation dataset, authentication, a frontend, or deployment. These were intentionally outside the required scope. Next I would add OCR for image-only PDFs, build a small human-labeled benchmark of job/resume pairs, measure accuracy or rank correlation against that benchmark, and reduce LLM calls by batching criterion scoring while checking that calibration remains stable.

Honest status: The core workflow is implemented, but this is not presented as a production-grade hiring system. The main remaining uncertainty is measured accuracy on a larger labeled dataset.